E-commerce · Web Development · Ukraine

GKit — E-commerce Website Development

From branded fashion storefront to scalable product-data architecture

OpenCart | CRM integration | SEO-ready launch

Project role

Development + technical architecture + post-release SEO foundation

Market
Ukraine

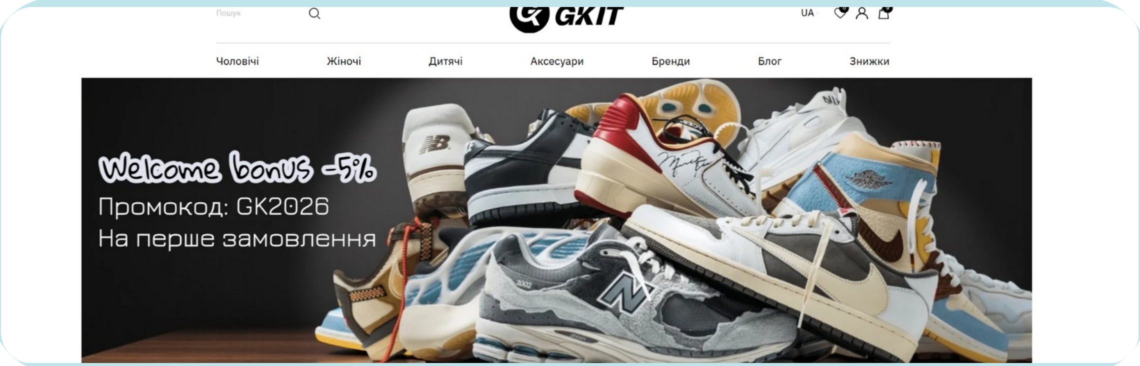
Niche
Fashion retail

Fashion-grade visuals, commerce-grade architecture

Client Context

A Ukrainian storefront for original branded fashion

GKit sells branded clothing, footwear and accessories for men, women and kids, with delivery across Ukraine and a 14-day exchange/return promise.



Original-only positioning

The store emphasizes verified suppliers and does not sell replicas or copies.

Broad catalog structure

Shoes, clothing and accessories across men’s, women’s and kids’ categories.

Service trust layer

Payment options, Ukraine-wide delivery, size consultation and returns were part of the retail experience.

Source: GKit website and ITForce public case study.

The Challenge

The website had to be more than a storefront

The team needed to launch a clean retail experience while solving product-data complexity, CRM synchronization and SEO-readiness from the start.

UI gaps

Language dropdown state, cart/wishlist counters and product review mechanics were missing in the design.

Catalog complexity

The initial product model risked creating duplicated product cards for each size and color.

CRM as source of truth

Inventory, product data and updates needed a reliable CRM → website logic.

SEO from launch

URLs, filters, metadata, indexing rules and analytics had to be considered before release.

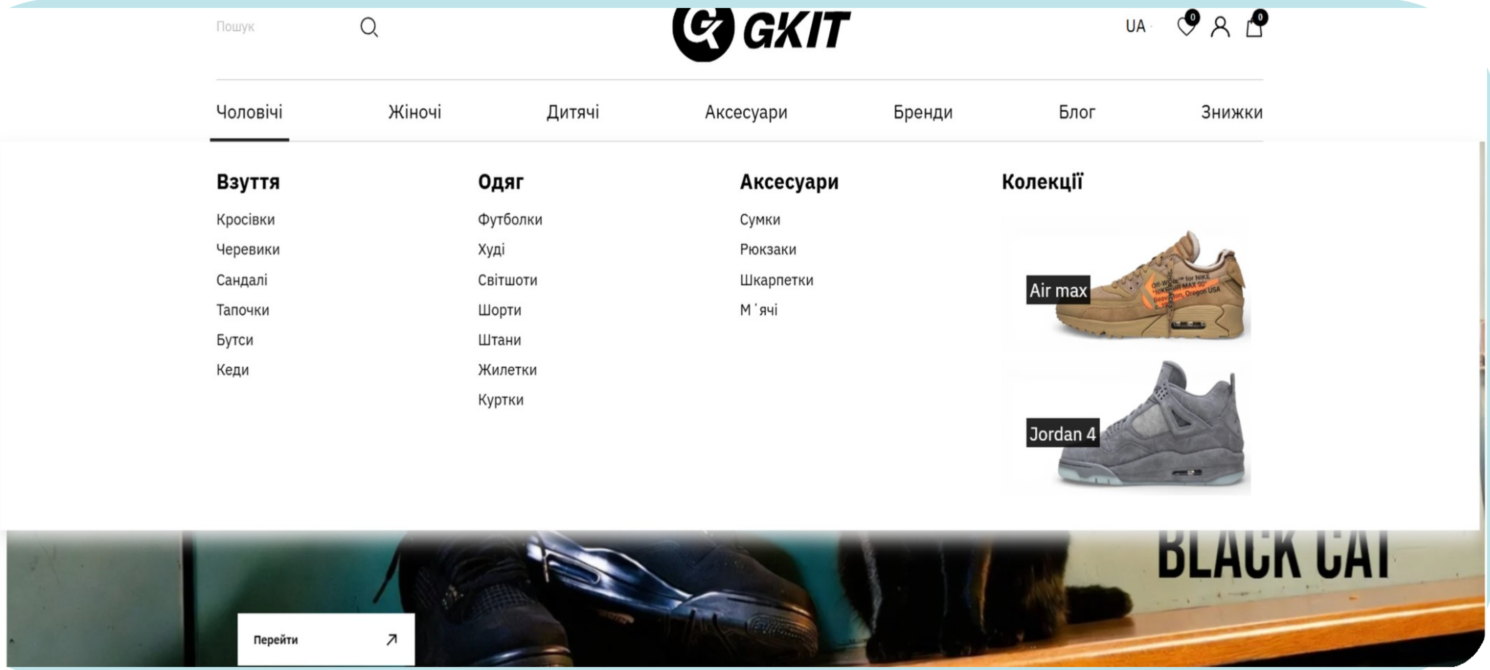
Core development thesis

For fashion e-commerce, the visible interface is only the top layer. The real value comes from data structure, operations flow and technical foundations that let the store scale.

Phase 1

OpenCart development

The project was implemented on OpenCart. T
the design early, identified missing UI states a
correct rendering across devices.



Responsive storefront

Cross-device rendering and layout logic reviewed before implementation.

Header states

Wishlist and cart counters added to make product activity visible without extra navigation.

Product feedback

Product card review mechanics defined so users could leave feedback on the product page.

Why it matters

Small missing states often become launch blockers in e-commerce. Catching them during development reduces rework and keeps the user journey consistent.

Architecture

Product data architecture: CRM → site

KeepinCRM source data → **API module + import rules** → **OpenCart catalog** → **Product pages + filters**

Required fields

SKU, model, stock, price and product name to protect the import from broken items.

Localized content

UA/RU product names, descriptions, characteristics and brand values.

Options

Size and color fields become product options and variations.

Filters

Delivery, type, collection, season and purpose create structured catalog filters.

Architecture principle

CRM stays the operational source of truth. The website receives clean structured data instead of manually maintained duplicate product entries.

Phase 2

From duplicate products to parent product + variations

Before

Instead of creating separate product cards for each size, the proposed model grou[illegible]s into variations under one parent product.

Sneaker 40

Sneaker 41

Sneaker 42

Sneaker 43

Bloated database
More filtering noise
Harder stock control

After

Parent product
Air Jordan model

Variations: 40 · 41 · 42 · 43 · 44

Color / size options on one product page

UX benefit

Customers choose size on one product page instead of jumping between duplicate listings.

Operations benefit

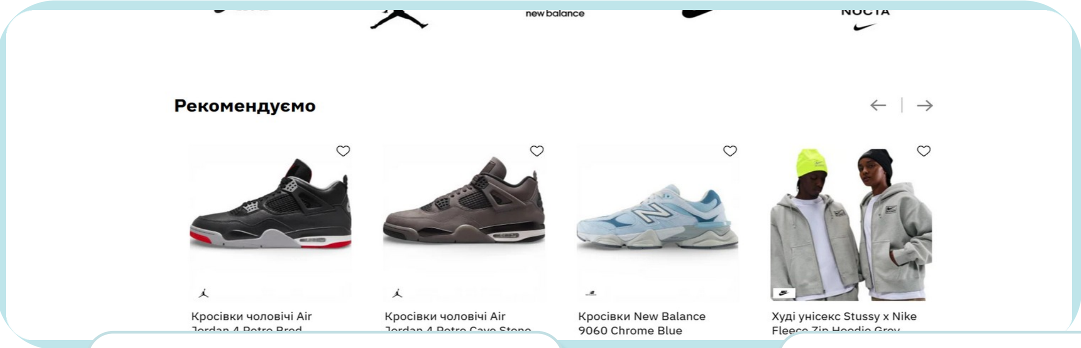
Stock tracking becomes cleaner because SKUs live under a structured parent model.

Development capability demonstrated

We can translate messy retail data into an architecture that improves catalog cleanliness, user experience and future SEO scalability.

Integration

Website → CRM order flow



The reverse flow was defined so that every completed order creates a CRM ord … for operationa…

Customer checkout → **Order payload** → **CRM order created automatically** → **Manager processing**

Products

List of ordered items

Commercials

Quantity and price

Customer

Buyer contact data

Fulfillment

Delivery and payment method

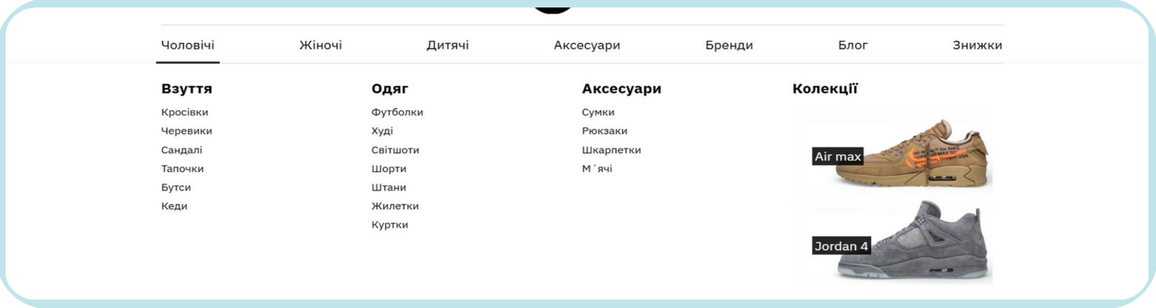
Why it matters

A fashion store can scale only when sales data, inventory and fulfillment move through one operational loop instead of being copied manually.

UX / UI

Catalog UX built for retail browsing

N
c



Brand trust

Popular brand tiles and brand-focused browsing reinforce the “original products” positioning.

Category depth

Men’s, women’s, kids’ and accessories sections were separated into clear shopping paths.

First-screen sales triggers

Bestsellers and brand carousel placement were recommended to increase CTR to product cards.

Sneakers | Hoodies | Brands | Collections | Blog | **Sale**

Performance

Media delivery and speed thinking

For image-heavy fashion retail, the team recommended a CDN-based media architecture: original photos in cloud storage, optimized delivery to the user and lightweight product pages.

Responsive image sizes

CDN can serve smaller images to phones and larger images to desktop users.

Modern formats

WebP / AVIF recommendations reduce file weight without visible quality loss.

Lazy loading

Non-critical images load when needed instead of blocking first render.

Caching resilience

Browser cache and CDN settings protect speed and stability.

Capability message

We do not treat product images as “just content.” For e-commerce, media delivery is part of conversion performance and SEO readiness.

Image weight target

-30–50%

potential reduction via WebP/AVIF recommendation

SEO-ready Build

Technical SEO was designed into the development checklist

HTTPS + 301 redirects

Single secure version of every page.

Clean URL rules

Hyphens, no Cyrillic, no spaces, consistent trailing slash.

Hreflang + lang="uk"

Correct language signals for multilingual pages.

Schema.org

Product, BreadcrumbList and Organization markup.

Canonical logic

Filter pages and duplicates controlled with canonical rules.

Noindex zones

Cart, checkout, account and search pages closed from indexing.

Sitemap + robots.txt

Post-fix sitemap and robots rules prepared.

Metadata templates

Optimized templates implemented for search snippets.

Development implication

SEO-readiness was not a separate “after launch” patch. It shaped URL, data, media, metadata and indexing decisions during development.

Launch

Launch readiness and first organic signals

After release at the end of February, the team ran pre-release and post-release che[illegible]nd delivered technical im[illegible]ements. The new site start[illegible]enerating org[illegible]visibility during the first m[illegible]

1. **Prototype audit**
2. **CRM data structure**
3. **Integration completed**
4. **Final client audit**
5. **Release + SEO checks**
   First month after release

Organic sessions	Engaged sessions	Engagement rate
85	**50**	**58.82%**

Avg interaction	Search clicks	Impressions
1 min	**42**	**2.6K**

Interpretation

For a brand-new site, early clicks, impressions and engaged sessions indicated that the technical foundation and snippet strategy were already being picked up by search engines.

Capabilities

Web development capabilities demonstrated

E-commerce builds
OpenCart storefront implementation and launch support.

CRM integrations
Product import logic and website-to-CRM order flow.

Data automation
SKU, model, stock, price, options and filter generation.

UX/UI audits
Design-state review, navigation logic and product card improvements.

Catalog architecture
Category, brand, collection and SEO-friendly filter structure.

Performance guidance
CDN, responsive images, WebP/AVIF and lazy loading.

SEO-ready code
URLs, hreflang, schema, metadata, sitemap and robots logic.

Analytics setup
GSC and GA4 launch-readiness tasks for measurement.

Build scalable e-commerce foundations for brands that need clean operations and search-ready gro

Let’s build

GKIT

Case source: ITForce public article + GKit website screenshots